

Kafka Failure Scenarios – Structured Notes

1. Active Controller Failure

Initial State

- Controller1: Leader
- Controller2: Follower
- Controller3: Follower

Failure & Election Process

- 1 Controller1 crashes.
- 2 Controller2 and Controller3 stop receiving heartbeat from the leader.
- 3 Election begins.
- 4 Controller2 starts election and requests votes.
- 5 Controller3 votes for Controller2.
- 6 Controller2 wins majority (2/3) and becomes the new leader.
- 7 Controller3 remains follower.

Impact: New leader is elected and cluster continues normally.

2. Leader Partition Failure

Setup

- Partition 0 (Replication Factor = 3)
- Leader: Broker1
- Follower: Broker2
- Follower: Broker3

Failure Process

- 1 Broker1 (leader) crashes.
- 2 Controller detects leader failure due to missing heartbeat.
- 3 Controller triggers leader election.
- 4 Candidates are Broker2 and Broker3 (both in ISR).
- 5 Broker2 becomes the new leader.
- 6 Controller updates metadata (Leader: Broker1 → Broker2).
- 7 ISR becomes {Broker2, Broker3}.
- 8 Producers and consumers receive metadata update.
- 9 Clients reconnect to Broker2 and system resumes operation.

Impact: No data loss. Automatic recovery.

3. Follower Partition Failure

Setup

- Partition 0 (Replication Factor = 3)
- Leader: Broker1
- Follower: Broker2
- Follower: Broker3

Failure Process

- 1 Broker3 crashes.
- 2 Leader (Broker1) detects Broker3 is unresponsive (no fetch requests).
- 3 Controller removes Broker3 from ISR.
- 4 ISR changes from {Broker1, Broker2, Broker3} to {Broker1, Broker2}.
- 5 System continues normally; producers still write and consumers still read.
- 6 Broker3 eventually recovers.
- 7 Broker3 catches up with leader data.
- 8 Broker3 is added back to ISR.

Impact: No data loss and no downtime.

4. Topic Failure

In Kafka, a topic itself does not fail because it is only a logical category used to organize messages. The real components that can fail are partitions or brokers. Therefore, failures occur at the partition or broker level rather than at the topic level.

5. Consumer Failures

Scenario 1: Consumer crashes before committing offset

- 1 Consumer polls events (offsets 100–199).
- 2 Consumer processes events 100–150.
- 3 Consumer crashes before committing the offset.
- 4 Consumer restarts.
- 5 Consumer asks Kafka for the last committed offset.
- 6 Kafka returns offset 99.
- 7 Consumer reprocesses events 100–150.

Result: At-least-once delivery (duplicate processing possible).

Scenario 2: Consumer heartbeat timeout

- 1 Consumer polls events.
- 2 Consumer starts processing which takes a long time (e.g., 6 minutes).
- 3 `max.poll.interval.ms` (default 5 minutes) is exceeded.
- 4 Group coordinator considers the consumer dead.
- 5 Rebalance is triggered.
- 6 Partitions are reassigned to other consumers.
- 7 Original consumer finishes processing.

- 8 Original consumer attempts to commit offset.
- 9 Coordinator rejects commit because consumer is no longer in group.

Result: Wasted processing.

Solutions

- Increase max.poll.interval.ms.
- Reduce max.poll.records to process smaller batches.
- Optimize processing logic.
- Use a separate thread pool for processing.

6. Broker Failure

Setup

- Cluster: 3 Brokers
- Topic: order-events
- Partitions: 6
- Replication Factor: 3

Failure Process

- 1 Broker1 crashes.
- 2 Controller detects broker failure due to missing heartbeat.
- 3 For partitions where Broker1 was leader, a new leader is elected from ISR.
- 4 Metadata is updated across the cluster.
- 5 For partitions where Broker1 was follower, it is removed from ISR.
- 6 Partitions continue operating with remaining replicas.
- 7 Clients receive updated metadata.
- 8 System continues normal operation.

Impact: No data loss and automatic recovery.